

Hybrid AI Risk Insurance

생성형 AI 시대의 새로운 보험 설계 프레임워크

01

AI 리스크

정의 및 발생 배경
AI 리스크의 유형
보험 가능성 검토

02

Hybrid AI-Insurance Model

AI 리스크 계량화: 심도-빈도 모델 활용
민감도 분석
보험료 지급

03

기대효과

가격 결정 주도권 확보
시장 수요 확대
신상품 경쟁력 제고

01

발생 배경: AI 도입의 급증

- 미국 기업 95%가 생성형 AI 업무 도입 (Bain & Company, 2025)
 - 전세계 생성형 AI 시장 수요:
2023년 \$106B → 2030년 \$2,110B, 약 20배 성장 전망
연평균 약 53.3% 성장률 (JEITA, 2024)
- 단기간에 기업 전반 침투, 보편적 업무 도구로 빠르게 확산

<그림 1> 생성형 AI시장의 수요 전망



자료 : 전자정보기술산업협회(JEITA)

02

AI 리스크 정의

- 인공지능 시스템의 자율성·비결정성·
학습 기반 구조로 발생하는 예측 불가능한 손해
- 특히 생성형 AI의 경우 확률 기반 출력, 블랙박스 모델 구조,
대규모 외부 데이터 학습이라는 특성으로 인해
기존 보험 약관 체계에서 정의되지 않은 새로운 위험 발생
→ 이러한 위험을 AI 리스크라 정의

AI 리스크 주요 유형 (1): 의사결정 오류

정의와 발생 원인

- AI 분석·추천 결과 기반 의사결정의 오류로 재무적 손실 발생
- 편향된 데이터, 모델 설계 오류, 통계적 과적합이 주요 원인
- 학습 데이터 편향 시 의사결정 판단에 바이어스 발생
- 금융배제나 사회적 신뢰 훼손으로 이어질 수 있음

IMF 보고서: 금융분야 AI 리스크

- 2023년 8월 IMF 보고서 발표
- AI 활용이 금융업 새로운 리스크 온상 가능성 지적
- 내재된 편견으로 인한 금융배제 위험 강조
- 금융분야 AI 관련 리스크 체계적 정리

의사결정 오류의 파급 효과

- 기업의 전략적 의사결정 왜곡 가능
- 대규모 투자 손실 및 사업 실패 위험
- 고객 신뢰 하락 및 브랜드 이미지 손상
- 규제 당국의 제재 및 법적 책임 발생

EY 조사: 글로벌 기업 손실 규모

- 글로벌 기업 975곳 대상 조사 실시
- 약 44억 달러(약 6조원대) 규모 손실 발생
- AI 결과 오류, 편향, 규제·컴플라이언스 문제가 원인
- AI 도입 리스크의 실질적 재무 영향 입증

AI 리스크 주요 유형 (2): Hallucination

01

Hallucination 개념

- 생성형 AI가 그럴듯한 허위 생성
- 확률 기반 모델의 구조적 위험
- 예측 불가능한 허위 정보 발생

02

구조적 위험 요인

- 확률 기반 출력 메커니즘의 한계
- 블랙박스 모델 구조의 불투명성
- 대규모 학습 데이터의 불확실성
 - 사실 검증 메커니즘 부재

03

허위 정보 생성

- 출처 왜곡 및 허위 정보 생성
- 존재하지 않는 판례 인용
 - 허위 계약 조항 생성
 - 잘못된 재무 수치 보고

04

실제 사례: Mata v. Avianca

- 미국 법정에서 발생한 실제 사건
- 변호사가 ChatGPT 생성 허위 판례 사용,
검증 없이 법정 문서에 제출
 - 법원으로부터 제재 조치

AI 리스크 주요 유형 (3):

자동화 시스템 오작동

01

자동화 시스템 오작동

- 로보어드바이저, 자동 심사, 자동 거래 시스템 적용
- 시스템 오작동 또는 통제 실패 발생

02

오작동 파급 효과

- 대량 금융 거래 오류 발생
- 보험 인수 자동심사 오류 및 제조 자동화 사고

AI 리스크 주요 유형 (4): 저작권 침해

AI 모델 학습 과정의 저작권 침해

- AI 모델 학습 시 저작권 보호 콘텐츠 무단 사용
- 저작권 침해 소송 및 집단소송 위험 증가
- 생성 콘텐츠의 저작권 귀속 불확실성
- 손해배상금 및 합의금 관련 재무적 리스크

국내 사례: 지상파 3사의 오픈AI 소송

- KBS, MBC, SBS 지상파 3사가 오픈AI 소송 제기
- 챗GPT 학습에 뉴스 콘텐츠 대량 무단 사용 주장
- 방송사 핵심 자산인 뉴스 콘텐츠 서비스 노출
- 지식재산권 침해에 대한 법적 책임 추궁

글로벌 사례: Disney·Universal vs Midjourney

- 2025년 6월 Disney와 Universal이 Midjourney 소송
- AI 이미지 생성 시 대표 캐릭터 무단 생성
- 저작권 침해로 인한 손해배상 청구
- 글로벌 엔터테인먼트 산업의 AI 리스크 대응

글로벌 사례2: 음악 산업의 AI 저작권 소송

- 소니·유니버설뮤직·워너가 Suno와 Udio 소송(2024)
- AI 음악 생성 기업의 저작권 침해 문제
- 음원 학습 데이터 무단 사용 논란
- 음악 산업 전반의 AI 저작권 이슈 확산

보험 가능성 검토: 보험 가능한 위험의 6가지 요건

우연성(Fortuity)

- AI의 자율성·비결정성·확률 기반 출력 특성
- 동일 입력에도 다른 결과 생성 가능
- Hallucination 등 예측 불가능한 확률적 사건
- 우연성 요건 충족

측정 가능성(Measurability)

- Stanford 'AI Index Report'(매년 발간) : AI 사고·오용 사례 데이터 수록
- PL보험·사이버보험 손해 통계 일부 활용
- 점진적 계량화 가능성 확보

동질성(Homogeneity)

- 산업군별 위험군 분류(금융·의료·제조)
- 사용 목적별 분류(의사결정·생성형·자율)
- 모델 규모별·통제수준별 분류
- ISO/IEC 23894:2023 국제 표준 활용

손해의 명확성(Definite Loss)

- 의사결정 오류: 재무 손실액으로 산정 가능
- Hallucination: 손해배상액으로 확정 가능
- 자동화 오작동: 사고 발생 시점·피해액 명확
- 저작권 침해: 손해배상 청구액 기준 손해 확정

손해의 대량성 (Large Numbers Law)

- 다수의 노출 단위가 존재해야 통계적 예측 가능.
- 전세계 조직의 88%가 최소 한 가지 이상의 업무 기능에 AI를 도입. (McKinsey & Company, 2025)
- 대수의 법칙 적용가능한 충분한 위험 집단 형성중

도덕적 위험 통제

- Human-in-the-loop 의무화
- 로그 보관 의무, 설명가능성 확보
- 외부 감사 의무 등 거버넌스 요건 약관 반영
- EU AI Act(2024) 고위험 AI 규제 활용
- 규제·표준·감사 체계를 활용한 언더라이팅 통제

STEP 1: 상품 구조 정의 및 유의점

1.보험료의 표준 구조

- Premium = 순보험료 + 위험부담금 + 비용 + 자본비용
- 순보험료(Pure Premium) = E[S]
- 기대손해를 기반으로 산정
- 각 구성요소의 명확한 정의 필요

2. 순보험료와 기대손해의 관계

- 순보험료 = 기대 총손실 E[S]
- 기대손해가 보험료의 핵심
- 빈도와 심도의 곱으로 계산
- AI 사고 특성 반영 필수

3. 변동성 및 Tail 위험 보상

- 변동성 보상의 정교한 반영
- Tail 위험 프리미엄 산정
- 극단 손해 대비 자본 적립
- 보험료 결정의 핵심 요소

STEP 2: Exposure 정의

담보별 Exposure 측정 기준

- AI 출력 오류: API 호출량
- 자동 의사결정 실패: 의사결정 건수
- 개인정보 유출: 보유 데이터 건수
- 학습 데이터 침해: 학습 데이터 규모

API 호출량 및 의사결정 건수

- API 호출량은 AI 출력 오류의 노출 지표
- 의사결정 건수는 자동화 실패 위험 측정
- 호출량과 건수가 많을수록 사고 발생 가능성 증가
- 담보 유형별 차별화된 측정 필요

데이터 규모 기반 측정

- 보유 데이터 건수는 개인정보 유출 위험 지표
- 학습 데이터 규모는 침해 위험 측정 기준
- 데이터 규모가 클수록 노출도 증가
- 데이터 보안 수준과 연계 평가

빈도 모델의 입력 변수

- Exposure는 빈도 모델의 직접 입력 변수
- $\lambda = \text{Exposure} \times q$ 구조
- 기업별, 담보별 차별화된 측정
- 사고 발생 확률 산정의 기초

빈도 모델링 — Poisson 기본

Poisson 모델은 사고 발생 빈도를 모델링하는 기본 구조로, $\lambda = \text{Exposure} \times q$ 형태로 기대빈도를 산정하며 $N \sim \text{Poisson}(\lambda)$ 분포를 따른다.

1 Poisson 모델의 기본 구조

- $\lambda = \text{Exposure} \times q$
- $N \sim \text{Poisson}(\lambda)$
- 평균과 분산이 동일하다는 가정
- 기대빈도 기반 사고 건수 예측
- 기본 모델로 활용

2 AI 사고의 과산포 특성

- Poisson 가정 충족하지 않을 수 있음
- AI 특성상 $\text{Var} > \text{Mean}$ 가능성 높음
- 과산포 반영 필요성 대두
- Negative Binomial 모형을 통한 검토
- 분산 조절 파라미터 도입

STEP 3: 빈도 모델링 (2) - 과산포 반영

01

Negative Binomial 모형

- $N \sim \text{NegBin}(r, p)$
- $\text{Var}(N) = \lambda + (\lambda^2)/k$

02

분산 조절 파라미터

- k = 분산 조절 파라미터
- 산업별 보정 적용

03

빈도와 심도의 독립성

- 공통충격 및 상태 변수가 주어졌다는 조건 하에서 빈도와 심도는 서로 독립이라고 가정

STEP 3: 빈도 모델링 (3)

상태전환 모델 추가

AI는 정상 상태와 Drift 상태를 가지며, Drift는 기업 내부 모델 성능 저하로 인해 사고 발생 확률이 구조적으로 증가하는 상태를 의미합니다. Drift 상태에서는 오류율 상승으로 사고 발생 확률이 구조적으로 증가하므로 기대빈도를 배율(m) 형태로 조정합니다.

Drift 시 기대빈도 배율 조정 및 모델 성능 저하에 따른 위험 증가

$Z \in \{0, 1\}$ (0=정상, 1=Drift), $P(Z=1)=p$

$\lambda(Z) = \lambda \times m$ ($m = 1$ or $m > 1$)

정상: 학습 안정·오류율 낮음; 비정상: 분포 변화·모델 노후화·업데이트 실패 등
비정상: 데이터 분포 변화, 모델 노후화, 업데이트 실패, 외부 환경 급변

STEP 3: 빈도 모델링 (4) - 공통충격 반영

01

공통충격(C)의 개념

공통충격(C)은 동일 모델·플랫폼 오류 등 시스템 수준의 동시다발 위험을 의미
변수 C 도입:
 $C \sim \text{Bernoulli}(\theta), C \in \{0,1\}, P(C=1)=\theta$

02

동시다발 사고 구조

유사 모델·SaaS 등 사용 기업 간 동시 사고 가능.
공통충격 변수 C 도입으로 동시다발 손해 구조를 반영

03

시스템 레벨 오류

C=1일 때 시스템 레벨 오류 발생
C=0일 때는 정상상태 가정
충격 발생 시 빈도가 증가하는 구조

04

독립 가정 완화

기업 간 사고는 기본적으로 독립으로 가정
공통충격(C) 도입으로 시스템 수준의 상관 위험을 반영

사고 건수의 최종 분포

AI는 과산포 가정하에 $N | (Z, C) \sim \text{NegBin}(\lambda(Z, C), k)$ 가정
k는 담보별로 추정하며 기본 λ 에 Drift(Z)와 공통충격(C) 배율을
반영해 조정된 $\lambda(Z, C)$ 를 구성
빈도와 심도는 기본적으로 독립이나 공통충격 시 동시 영향이 가능

STEP 4: 심도 모델링 (1) - Body + Tail 구조

1. AI 사고 손해의 기본 구조

- 대부분 소액·중간 손해(Body) 발생
- 규제·소송·대규모 장애 시 극단 손해(Tail) 확대
- 두 유형을 구분해 복합 구조로 모델링

2. 일반손해(Body): Lognormal 분포

- $X_{body} \sim \text{Lognormal}(\mu, \sigma)$
- 복구비·대응비·중소 배상 등 일반적인 손해 포함
- 로그노말 분포는 손해액이 음수가 될 수 없고 오른쪽으로 치우친 분포 특성에 적합

3.극단 손해(Tail): Pareto 분포

- $X_{tail} \sim \text{Pareto}(\alpha, x_m), 2 < \alpha < 3$
- 집단소송·대규모 과징금·광범위 서비스 장애 포함
- Pareto 분포는 대형 손해의 두꺼운 꼬리(Heavy Tail) 구조 반영

4. 임계값 분리 및 기대값 계산

- 임계값: $u = Q_{0.95}(X)$
- Tail 확률: $\pi = P(X > u)$
- 기대값: $E[X] = (1 - \pi)E[X_{body}] + \pi E[X_{tail}]$
- 임계값으로 Body/Tail 분리 후 각각 기대값 산정

5. Body + Tail 복합 구조의 의미

- AI 보험의 손해 구조는 일반 손해와 극단 손해의 복합 형태로 구성
- 평시에는 Body 중심이나 사건 발생 시 Tail로 전환 가능
- 이러한 복합 구조는 AI 리스크의 이중적 특성을 정교하게 반영하며, 보험료 산정 시 Tail 위험 관리가 핵심 요소로 작용합니다.

STEP 4: 심도 모델링 (2) - 사건(event) 효과

1. 사건 발생 시 Tail 확률의 증가

- AI 사고는 평소에는 운영비용 중심이나 규제 조사·집단소송·저작권 분쟁 등 사건 발생 시 대형 손해로 전환
- 본 모델은 사건 발생 시 Tail 확률이 상승한다고 가정
- 평시와 사건발생 시의 손해 구조 차이를 명확히 반영

2. 사건 효과 수식 구조

- $\pi' = \pi_{base}(1 + \rho\kappa)$
- π_{base} : 평소 대형 손해 발생 확률
- ρ : 사건 발생 확률(담보별 상이)
- κ : 사건 발생 시 Tail 확대 강도
- 사건 반영 후 최종 Tail 확률 산출

3. 담보별 사건 구조 및 Tail 민감도 차등 설계

의사결정 오류: 규제·차별 이슈로 사건 가능성이 높아 Tail 비중(π), 사건 확률(ρ)을 상향 적용

Hallucination: 평시 손해는 제한적이나 사건화 시 규모 점프 가능성이 커 Tail 확대 계수(κ)를 높게 적용

설계 원칙: 담보별 사건화 구조에 따라 ρ 와 κ 를 차등 적용하여 Tail 민감도 중심으로 설계

STEP 4: 심도 모델링 (3) - 공통충격 효과

01

공통충격 시 심도 스케일 확대

업데이트 이슈 발생 시 여러 계약자 동시 영향
공통충격은 손해를 배율로 확대

02

손해 규모 배율 확대 수식

$$X_{\text{shock}} = X \times (1 + \gamma)$$

γ 는 공통충격 시 평균 손해 확대율

03

담보별 Tail 특성 차등 적용

자동차 오작동은 장애 확산으로 γ 높음: 학습데이터
침해는 빈도 낮지만 사건화 시 대형 Tail 발생

STEP 5-6: 기대손해 계산 및 보험 조건 반영

기대손해는 사고 발생 빈도와 사고당 손해 규모의 곱으로 계산되며, 이는 AI 보험료 산정의 핵심 출발점이다.

1

빈도와 심도의 결합: $E[S] = E[N] \times E[X]$

- 빈도($E[N]$): 기본사고율에 Drift·Shock 반영으로 증가
- 심도($E[X]$): Body+Tail에 Event·Shock 반영으로 대형손해 비중 증가
- 공통충격은 건수와 규모를 동시에 증폭
- 최종: (기본빈도×Drift×Shock) × (기본심도×Event×Shock)

2

보험조건 반영: Limit 및 Deductible

- $E[X_{insured}] = E[\min(X, L)]$: Limit로 지급 상한 적용
- $E[X_{net}] = E[\max(X - d, 0)]$: Deductible로 소액 제외
- 보험조건은 Tail 노출 조정에 핵심적 역할 수행

STEP 7-8: 총손실 분산 및 자본 요구량

01

총손실 분산 계산식

- $\text{Var}(S) = E[N]\text{Var}(X) + \text{Var}(N)(E[X])^2$
- 총손실 분산은 빈도·심도 변동성 동시 반영
- 빈도·심도 변동성 동반 시 $\text{Var}(S)$ 급증

02

분산 급증 요인

- $\text{Var}(N)$ 큼: 과산포·드리프트로 빈도 변동성 ↑
- $\text{Var}(X)$ 큼: Heavy Tail·이벤트로 심도 변동성 ↑
- $\text{Var}(S)$ 급증 시 Risk Load·자본 요구량 상승

03

자본 요구량 산정

- $\text{Required Capital} = \text{VaR}_{99-99.5\%}(S) - E[S]$
- 평균 손해 외 극단 손해 대비 자본 적립 필요
- 총손실 상위구간 기준으로 자본 산정
- VaR 기준은 지급능력 확보용

04

AI 특성 반영

- Tail 비중 ↑ → VaR 급증
- 사건화 시 자본 요구 증가
- Tail 중심 자본 관리가 핵심
- 극단 손해 시 지급능력 확보가 필수

STEP 9-10: Risk Load 및 최종 보험료 구조

1. Risk Load (위험부담금): 변동성 보상의 핵심

Risk Load는 평균 손해에 더해지는 변동성 보상으로, $\text{Risk Load} = k \cdot \sigma$ 공식으로 계산된다. 여기서 σ 는 연간 총손실의 표준편차이며, k 는 위험가중계수이다. AI 보험의 경우 과산포 빈도와 Heavy Tail 심도로 인해 총손실 변동성이 크게 나타나므로, 표준편차 기반 변동성 보상 구조를 적용한다. 이는 보험 인수 위험에 대한 프리미엄으로 작용하며, 변동성이 클수록 Risk Load가 증가하는 구조이다.

2. 최종 보험료 구조: $\text{Premium} = E[S] + \text{Risk Load} + \text{Expense} + \text{CoC}$

최종 보험료 = $E[S] + \text{Risk Load} + \text{Expense} + \text{CoC}$

AI 보험은 평균보다 Tail 위험·변동성 관리가 가격 핵심

보험료는 극단 손해에 대한 자본 부담을 반영해야 함

보험료 민감도 분석 (1) - 분석 목적 및 주요 변수

1. 분석 목적: AI 리스크의 불확실성 관리

- AI 리스크는 과거 통계 불충분, 사건화·대형손해 영향 큼
- π, ρ, α 세 변수 중심 민감도 분석 수행

2. 대형 손해 비중(π) 변화의 영향

- 평시엔 소규모 손해, 사건화 시 대형손해 비중 ↑
- 보수($\pi \uparrow$), 기준, 낙관($\pi \downarrow$) 시나리오 설정
- π 증가 시 평균손해 완만 ↑, VaR·자본은 비선형 확대
- AI 보험은 Tail 비중 변화에 매우 민감

3. 사건 발생 확률(ρ) 변화의 영향

- ρ 는 규제 조사, 집단소송 등 사건이 발생할 확률을 의미
- ρ 증가 시 평균 손해는 선형적으로 증가하나 자본 요구량은 급격히 증가
- AI 보험의 가격 결정이 '사건 가능성'에 크게 좌우됨을 의미
- 사건화 가능성이 높은 담보(의사결정 오류 등)는 보험료 할증 필요

4. Pareto 파라미터(α) 변화의 영향

- AI 산업은 Heavy Tail 구조를 가지므로 α 값 변화에 따라 Tail 두께가 달라짐
- α 감소($2.8 \rightarrow 2.3$) 시 평균 손해는 소폭 증가하나 VaR은 급증
- AI 보험의 자본 요구량은 Tail 두께에 매우 민감
- Tail 두께가 자본 요구량을 결정하는 핵심 변수임을 확인

분석 결과 및 시사점

AI 보험은 평균 손해보다 Tail 변화에 더 민감하다.

사건 확률 상승 시 자본 요구량이 급증하며, 민감 변수들은 평균보다 VaR·자본에 더 큰 영향.

따라서 Tail 중심 가격정책, 재보험·Aggregate Cap 및 공동충격 관리가 필수이다.

보험금 지급 설계 (1) - 단계별 보장 구조

1. AI 사고의 구조적 특성

- AI사고의 과정: 오류→인지→조사→규제→중단
- 소액다발 + 가끔 대형손해 가능
- 동일모델 문제 시 동시 확산, 단계별 보장 필요

2. 초기 대응 비용 보장

- 포렌식 조사비 보장
- 외부 전문가·법률 자문비 보장
- 초기 단계 신속 원인 파악 및 대응 지원

3. 규제·법적 대응 비용 보장

- 감독기관 조사 대응 비용 보장
- 소송 방어 비용 보장
- 규제 당국과의 커뮤니케이션 비용 포함
- 법적 분쟁 장기화에 따른 비용 부담 완화

4. 운영 중단 손실 보장

- 시스템 장애로 인한 서비스 중단 손실 보장
- 서비스 정지 기간 동안의 매출 손실 보상
- 복구 비용 및 긴급 대응 비용 포함
- AI 시스템 의존도가 높은 기업의 핵심 보장 항목

5. 대형 책임 손해 보장

- 집단소송 배상금 보장
- 규과금·법원 손해배상 보장
- Tail 손해 고려한 한도 설정 필요

보험금 지급 설계 (2) - Trigger 기반 구조

1. 계단형(Trigger-based) 지급 구조의 도입

- 손해 산정 분쟁을 줄이기 위해 지표 기반 지급 구조 도입
- Downtime 6시간 초과 시 정액 지급
- Hallucination rate 8% 초과 시 정액 지급
- 운영 리스크에 한정 적용

2. 손해 산정 분쟁 감소 효과

- 지급 명확성 확보로 분쟁 최소화
- 객관적 지표 기반 신속한 보험금 지급
- 책임 손해 영역에는 전통적 손해보상 구조 병행
- 투명한 보상 기준 제시

3. 도덕적 위험 통제 방안

- 명확한 Trigger 기준으로 도덕적 위험 통제
- 과도한 청구 방지 메커니즘
- 객관적 측정 지표 활용
- 보험 사기 예방 효과

보험금 지급 설계 (3) - Aggregate Cap 설정

01

사고당 한도 및 연간 총한도

- 사고당 한도(Limit) 명확히 설정
- 연간 총한도(Aggregate Limit) 적용

02

Cat Event 한도 설정

- Cat Event 기준 명확화
- 시스템 수준 사고 정의와 연계

03

공통충격 시 다수계약 동시 Trigger 관리

- 동시다발 사고 가능성 대비
- 다수 계약 동시 Trigger 발동 관리

선도 보험사의 AI 리스크 보험 가격 결정권

가격 리더십 확보 및 시장 진입 장벽 제고

- AI 리스크 평가 및 손해 사례 축적
- 언더라이팅 프레임워크 선제 구축
- 경쟁사 대비 가격 설정 주도
- '가격 리더' 역할 확보

경험적 데이터 기반 요율 설정 능력

- 경험적 데이터 기반 요율 설정
- 리스크 선별 능력 보유
- 시장 기준점 설정 가능
- 경쟁 우위 확보

리스크 커버리지 세분화 및 차별화

- 보장 범위 및 조건 정교화
- 높은 인수능력 확보
- 고유 리스크 풀 구축
- 재보험 시장 협상력 강화

신상품의 시장 경쟁력

AI 오류 보장 상품화 사례

- Lloyd's of London 실제 출시
- AI 챗봇 오류 손실 담보
- 상업상품으로 작동 중
- 기존 보험과 차별화된 특화 담보

강력한 기업 수요 존재

- 기업의 약 90%가 보장 원함
- AI/생성 AI 리스크 보험 수요
- 실수요 명확히 존재
- 보장 범위 확대 요구

프리미엄 지불 의향

- 67% 이상 추가 지불 의향
- 일반 보험료 대비 10% 더 지불
- 높은 가격 수용성
- 상품 개선 경쟁력 강화

AI 리스크 보험 시장의 성장성

01

빠른 시장 성장

기업의 AI 리스크 관련 보험 시장 급성장 중
, AI 활용이 확대될수록
기업 차원의 리스크 전이 수요가 증가

02

구체적 성장 규모

연평균 약 80% 성장률, 2032년에는 연간 보험료
규모 약 48억 달러(약 6조 원)에 이를 것으로 전망
(Deloitte, 2023)

보험사 및 기업 측면의 이점 정리

보험사 측면

- 가격 주도권 확보:
AI 리스크 전용 데이터, 언더라이팅 능력 확보 시
시장 기준점 설정 가능
- 새로운 수익원:
높은 성장률 시장에서 프리미엄 확대 가능성
- 차별화된 경쟁력:
전통 보험사 대비 'AI 보장 전문성' 보유

기업/피보험자 측면

- 실제 리스크 전이:
AI 오류 및 법적 책임 관련 손실 보장 가능
- 재무 안정성 강화:
AI 운영 리스크를 비용으로 전가해 경영 안정성 향상
- 시장 신뢰 확보:
규제와 법적 이슈 환경에서 AI 거버넌스 신뢰성 강화

```
$ cat project_summary.md
```

국내 주식 퀀트 시스템

가설 검증 · 방법론적 함정 발견 · 진입 필터 고도화 · 실전 자동매매 구현

PROJECT FILES

01_sde_theory_verification.py ← 이론 검증

02_kalman_ou_calibration.py

03_girsanov_verification.py

04_sanity_check_random_walk.py ← 핵심 발견

05_threshold_scan.py

06_portfolio_diversification.py

07_candle_volume_delta_filter.pine ← 진입 필터

08_ultimate_rsi_filter.pine

09_auto_trading_architecture.py

무엇을, 어떻게 검증했는가

WHAT - 무엇을 했는가

목표

국내 주식시장에서 통계적으로 유의한 반전(역추세) 신호를 찾고, 진입 타이밍을 캔들 단위 거래량·모멘텀 필터로 보강해 실전 자동매매 시스템까지 구축

접근

가설 → 백테스트 → 합정 의심 → 재검증을 반복하며, 통계적으로 살아남은 신호만 채택. 진입 타이밍은 공개 Pine Script 지표를 코드 단위로 분석해 필터로 결합

범위

이론 검증 → 전략 백테스트 → 임계치 스캔 → 포트폴리오 분석 → 캔들 거래량·모멘텀 필터 결합 → 실전 시스템화, 6단계

HOW - 무엇을 사용했는가

언어 / 라이브러리

Python - numpy, pandas, scipy (optimize, stats)
Pine Script v6 (TradingView) - 진입 필터 로직

통계 기법

최우추정법(MLE), t-검정, Ornstein-Uhlenbeck 과정 추정

검증 방법론

랜덤워크 sanity check, 표본 확장, 임계치 스캔, 공개 지표 코드 분석 후 임계치 재조정

// overview

가설 → 반박 → 재검증, 다섯 단계

01

가설 수립

±5% 급락 시 반전매매

누적수익 50.7%, 승률 65%

02

표본 편향 발견

신호 70%가 특정 기간 쏠림

결론 보류

03

방법론 자체 의심

spread=price-MA 방식 검증

가짜 신호 함정 발견

04

표본 확장

30종목 · 3,874개 사건

결론이 반대로 뒤집힘

05

데이터 기반 채택

5~10% 임계치 스캔

5~6%만 유의($t > 2$)

```
// 01_sde_theory_verification.py
```

실전 전에, 이론부터 수치로 검증



```
def verify_variance_property(n_paths=50000):  
    # Var(B_t) = t 를 몬테카를로로 검증  
    paths, dt = simulate_brownian_motion(n_paths)  
    empirical_var = paths[:, -1].var()  
    error = abs(empirical_var - T) / T  
    return error # 0.874%  
  
def gbm_analytic_vs_euler_maruyama():  
    # GBM 해석해 vs Euler-Maruyama 수치해  
    S_analytic = S0 * np.exp(  
        (mu - 0.5*sigma**2)*T + sigma*B_T  
    )  
    S_numeric = euler_maruyama_solve(...)  
    diff = abs(S_analytic.mean() - S_numeric.mean())  
    return diff # 0.002%
```

\$ 이론 검증 결과 (3개 항목)

브라운 운동 분산 성질

이론값=1.0000 실측값=0.9913 오차=0.87%

이토 등거리(Ito Isometry)

좌변=0.4883 우변=0.5010 오차=2.54%

GBM 해석해 vs 수치해 교차검증

해석해=104.819 수치해=104.817 오차=0.002%

이론을 배우는 데 그치지 않고, 실전 전략에 쓰기 전 먼저 시뮬레이션으로 수치가 맞는지 검증하는 습관을 다진 단계입니다.

```
// 04_sanity_check_random_walk.py
```

핵심 발견: 가짜 평균회귀 탐지



```
def run_sanity_check(n_paths=200, n_steps=1000, ma_window=20):
    kappas = []
    for i in range(n_paths):
        # 순수 랜덤워크 (신호 없음)
        random_walk = np.cumsum(
            np.random.normal(0, 1, n_steps)
        )
        ma = moving_average(random_walk, ma_window)
        spread = (random_walk - ma)[ma_window-1:]

        result = minimize(
            ou_negative_log_likelihood,
            x0=(2.0, 0.0, 1.0),
            args=(spread, 1.0),
        )
        kappas.append(result.x[0])

    t_stat = kappas.mean() / (kappas.std() / sqrt(n))
```

```
$ python 04_sanity_check_random_walk.py
```

```
kappa 평균 = 0.0815
표준편차    = 0.0112
t-stat      = 102.86
```

→ 신호가 전혀 없는 데이터에도
kappa가 유의하게 0이 아님
→ spread=price-MA 방식 자체가
가짜 신호를 생성 → 폐기

정교해 보이는 신호가, 실은 계산 방식 자체의 함정이었다는
것을 랜덤워크 시뮬레이션으로 직접 증명했습니다.

```
// 05_threshold_scan.py
```

데이터 기반 임계치 결정

```
def scan_thresholds(returns, thresholds):
    results = []
    for th in thresholds:
        res = event_study_t_test(returns, th)
        results.append(res)
    return results

def event_study_t_test(returns, threshold,
                       holding_days=10):
    signal_returns = []
    for i in range(n_stocks):
        for t in range(n_days - holding_days):
            r = returns[i, t]
            if abs(r) > threshold:
                fwd = returns[i, t+1:t+1+holding_days]
                signal_returns.append(-np.sign(r)*fwd.sum())
```

```
$ scan output
```

threshold	t-stat	결과
5%	4.67	유의
6%	3.67	유의
7%	3.45	유의
8%	2.64	유의
9%	1.19	기각
10%	1.25	기각

7% 이상부터 유의성 소실 → 5~6%를 실증적으로 최종 채택
(임의의 라운드 넘버가 아님)

```
// 06_portfolio_diversification.py
```

포트폴리오 분산효과 정량화



```
def portfolio_variance_reduction(
    deposit, stock, coin,
    weights=(0.4, 0.4, 0.2)
):
    returns = np.vstack(
        [deposit, stock, coin]
    )
    cov = np.cov(returns) * 252

    w = np.array(weights)
    port_var = w @ cov @ w
    port_vol = np.sqrt(port_var)

    # 자산별 위험기여도
    marginal = cov @ w
    risk_contrib = w * marginal / port_var
    return port_vol, risk_contrib
```

```
$ python 06_portfolio_diversification.py
```

주식-코인 상관계수 = 0.078
주식 단독 변동성 = 21.7%
4:4:2 포트폴리오 변동성 = 14.8%
변동성 감소 = 6.9%p

예금 40% → 위험기여도 0.0%
주식 40% → 위험기여도 37.7%
코인 20% → 위험기여도 62.3%

명목비중은 코인이 주식의 절반이지만, 실제 위험기여도는 코인이 더 큼니다.

```
// 07_candle_volume_delta_filter.pine
```

진입 타이밍 필터 (1) — 세력 개입 캔들 & 거래량 델타



```
// 세력 개입 캔들(Displacement) 판별
```

```
bodyRatio      = |close - open| / (high - low)
```

```
isDisplacement = bodyRatio ≥ bodyPct
```

```
highVol        = volume > sma(volume, len) × volMult
```

```
isShift        = (bull v bear) ∧ highVol ∧ isDisplacement
```

```
// 하위 타임프레임(1분봉) 매수/매도 체결량 분리
```

```
buyVol  = ∑ volume (lower-tf, close>open)
```

```
sellVol = ∑ volume (lower-tf, close<open)
```

```
deltaVol = buyVol - sellVol
```

```
buyRatio = buyVol / (buyVol + sellVol)
```

```
// 캔들 내부 매수/매도 블록 분할(단일 캔들 근사)
```

```
buyRatio_intra = (close - low) / (high - low)
```

\$ 지표 로직 요약 (2개 지표)

Institutional Displacement

→ bodyRatio·거래량 급증으로 “세력 개입 캔들” 표시

Delta Trading

→ 캔들 내 매수/매도 비중 계산 + EMA(12,26) 크로스를
거래량으로 재확인

반전매매 신호(z_t, OU)만으로는 “지금 진입해도 되는 타이밍인지” 보강할 장치가 부족해, 검증된 공개 지표의 계산 구조를 코드 단위로 분석한 뒤 진입 조건 앞단의 필터로 재구성했습니다.

```
// 08_ultimate_rsi_filter.pine
```

진입 타이밍 필터 (2) — 대량 매수·매도 & Ultimate RSI



```
// 대량 매수/매도 캡처 - 거래량 임계치(1.0~2.0배) 분류  
sma = sma(volume, len)  
f1..f5 = volume > sma × [2.00, 1.75, 1.50, 1.25, 1.00]
```

```
// Ultimate(Augmented) RSI - 노이즈를 줄인 모멘텀  
upper = highest(src, len) , lower = lowest(src, len)  
diff = (upper > upper[1]) ? r  
      : (lower < lower[1]) ? -r : (src - src[1])  
arsi = ma(diff, len) / ma(|diff|, len) × 50 + 50
```

```
// 반전매매 진입 필터 결합(AND 조건)
```

```
entry = reversal_signal(z_t, 0U)  
  ∧ isShift // 세력 개입 캔들  
  ∧ (buyRatio > threshold) // 체결 강도  
  ∧ (osValue < arsi < obValue) // 과매도 이탈 아님
```

\$ 필터 결합 결과

진입조건 4항목 AND 결합

→ “신호는 났지만 진입 타이밍은
아닌 경우”를 걸러내는 필터 계층

지표를 새로 만들지 않고 공개 코드를 분석하는 방식을 택
해, 그 시간을 “반전매매 전략 안에서 어떻게 해석·결합할
지”에 썼습니다. 임계치 파라미터는 본인이 주로 매매하는
종목군의 평균 거래량 패턴에 맞춰 재조정했습니다.

출처 및 저작권 — 위 지표는 TradingView 공개 Pine Script를 학습·분석해 파
라미터·활용 로직만 조정한 것으로, 원 코드 저작권은 각 원저작자에게 있습니다.
“Delta Trading”은 **hapharmonic**(MPL 2.0), “Filter RSI / Ultimate RSI”는 **RICH
TRADING SCHOOL**(CC BY-NC-SA 4.0)의 코드를 기반으로 합니다.

// 09_auto_trading_architecture.py

실전 자동매매 아키텍처



```
class RiskFilter:
    def passes_all_checks(self, now):
        if self.is_halted():
            return False # HALT.flag 감지
        if not self.is_trading_hours(now):
            return False # 09:00~15:30 외 스킵
        if self.daily_loss_exceeded():
            self._trigger_halt("일일 손실한도 초과")
            return False
        if not self.can_open_new_position():
            return False # 최대 10개 포지션
        return True
```

SignalScanner → RiskFilter → OrderExecutor →
PositionTracker → Logger

entry = z_t.00 신호 ∧ 07:08 지표 필터
→ 진입 조건 자체에 캔들·모멘텀 필터가 결합됨

개별 포지션 손절 -10% · 거래시간 09:00~15:30 · 에러 발생 시
즉시 전체정지 · 일일 손실한도 -25%

현재 모의투자 계좌로 스케줄러 등록, 30분 간격 실행 중

한계와 향후 보완 계획

다중검정 문제

임계치를 5~10%까지 여러 번 스캔한 것 자체가 다중검정(multiple testing) 상황입니다. Bonferroni 보정 등으로 유의수준을 더 엄격히 조정할 재검증이 필요합니다.

사건 간 독립성 가정

같은 시기에 여러 종목이 동시에 급락하면 사건들이 서로 독립이 아닐 수 있어, 표본 크기(3,874개)가 실제보다 과대평가됐을 가능성이 있습니다.

지표 필터 과최적화 위험

공개 지표의 거래량·RSI 임계치를 본인 종목군에 맞게 재조정했으나, 이 재조정이 과거 데이터에만 맞는 과최적화인지 별도의 out-of-sample 검증이 아직 필요합니다.

실전 전환 시점

모의투자에서 리스크 안전장치와 진입 필터가 실제 상황에서 의도한 대로 작동하는지 충분한 기간 관찰한 뒤, 실전 계좌 전환 여부를 판단할 예정입니다.

스스로 발견한 한계를 숨기지 않고, 다음 검증 단계를 구체적으로 설계하는 것도 검증 과정의 일부라고 생각합니다.

```
$ git log --oneline
```

배운 것

fix	표본 편향 발견 → 재설계
revert	가짜 평균회귀 함정 확인 → spread 방식 폐기
refactor	표본 30종목 확장 → 결론 반전 확인
feat	데이터 기반 임계치(5~6%) 채택
feat	공개 지표(거래량 델타·Ultimate RSI) 분석 후 진입 필터 결합
ship	안전장치 포함 자동매매 시스템 구축

데이터가 그럴듯한 결과를 보여줘도 표본 편향·과최적화·look-ahead bias를 항상 의심해야 한다는 것, 그리고 검증된 남의 코드를 가져다 쓸 때도 “왜 이렇게 계산되는지”를 직접 뜯어본 뒤에만 전략에 결합해야 한다는 것을 확인했습니다.